State Monad

State · Pure RNG / bank balance simulation · Build with Waqar

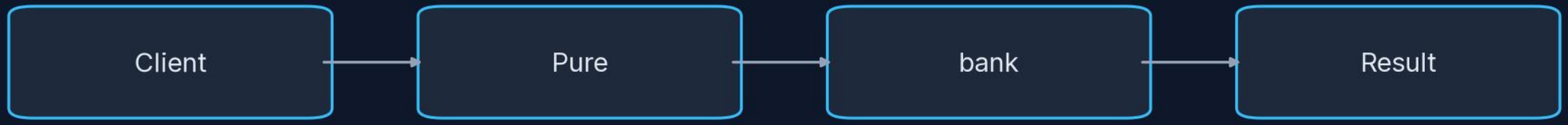
Today's Hook

- Problem: Pure RNG / bank balance simulation
- Pattern: State — earned from the pain, not trivia.
- Runnable main on JDK 25 with captured stdout.
- Naive → solution → failure paths → trade-offs.

The Problem

- Pure RNG / bank balance simulation
- Naive design shows the cost clearly.
- Pattern isolates the force behind a clean boundary.
- Tests cover happy path and injected failures.

Structure



State: Pure RNG / bank balance simulation

Real-World Scenario

- Running example: Pure RNG / bank balance simulation
- Same names in naive code, refactor, and Demo output.
- No shared StreamForge framing unless it fits.
- Product language first; pattern vocabulary second.

Naive Code

```java
package com.buildwithwaqar.patterns.functional.state;

import com.buildwithwaqar.patterns.functional.state.State;
/** Problem: simulate a bank balance purely (state threaded, not mutated globally). */
public final class UseCaseDemo {
  public static void main(String[] a) {
    System.out.println("=== State problem: pure bank simulation ===");
    State<Integer, Integer> deposit20 = State.<Integer>get()
      .flatMap(bal -> State.<Integer>set(bal + 20).flatMap(x -> State.pure(bal + 20)));
    var r = deposit20.run(100);
    System.out.println("returned=" + r.value() + " newBalance=" + r.state());
  }
}
```

Why Naive Hurts

- Call sites couple to the wrong shape or leak on partial failure.
- Parameter drilling / duplicated envelopes / missing rollback.
- Hard to unit-test without the real dependency.
- New variants require shotgun edits.

Solution

```java
package com.buildwithwaqar.patterns.functional.state;

import java.util.function.Function;

public record State<S, A>(Function<S, Result<S, A>> run) {
    public record Result<S, A>(S state, A value) {}

    public Result<S, A> run(S s) { return run.apply(s); }

    public <B> State<S, B> map(Function<? super A, ? extends B> f) {
        return new State<>(s -> {
            var r = run(s);
            return new Result<>(r.state(), f.apply(r.value()));
        });
    }

    public <B> State<S, B> flatMap(Function<? super A, State<S, B>> f) {
        return new State<>(s -> {
            var r = run(s);
            return f.apply(r.value()).run(r.state());
        });
    }
```

Solution (2)

```
    public static <S, A> State<S, A> pure(A a) { return new State<>(s -> new Result<>(s, a)); }
    public static <S> State<S, S> get() { return new State<>(s -> new Result<>(s, s)); }
    public static <S> State<S, Void> set(S s) { return new State<>(ignored -> new Result<>(s, null)); }
    public static <S> State<S, Void> modify(Function<S, S> f) {
        return new State<>(s -> new Result<>(f.apply(s), null));
    }
}
```

Runnable main

```java
package com.buildwithwaqar.patterns.functional.state;

import com.buildwithwaqar.patterns.functional.state.State;
/** Problem: simulate a bank balance purely (state threaded, not mutated globally). */
public final class UseCaseDemo {
  public static void main(String[] a) {
    System.out.println("=== State problem: pure bank simulation ===");
    State<Integer, Integer> deposit20 = State.<Integer>get()
      .flatMap(bal -> State.<Integer>set(bal + 20).flatMap(x -> State.pure(bal + 20)));
    var r = deposit20.run(100);
    System.out.println("returned=" + r.value() + " newBalance=" + r.state());
  }
}
```

Walkthrough Output

```
=== State problem: pure bank simulation ===
returned=120 newBalance=120
```

Step-by-Step Trace

- Each stdout line maps to a stage above.
- Re-run the main class locally on JDK 25.
- Failure-path lines prove compensations / rejects / missing files.
- Pause and predict the next line before revealing.

Modern Java 25 / Zazr

```
// Java 25 / Zazr direction
// records + sealed + switch
// inject Env / collaborators at the edge
// virtual threads for blocking fan-out
```

Trade-offs

- Clarity vs ceremony — refuse five types for two lines.
- Teaching demos omit production metrics/idempotency keys.
- Prefer simpler language features when they erase the force.
- Know the twin pattern often confused with State.

Related Patterns

- Contrast State with its neighbor pattern.
- Name the roles from memory in sixty seconds.
- Point to the season map for what comes next.
- Link the GitHub episode folder in the description.

Recap

- Problem → naive → State → Demo stdout.
- Failure paths are part of the lesson.
- Teach when NOT to use it.
- Practice: break a test, fix it, re-run the main.